Open With The Title

Good morning. The official title of this talk is Same AI, Different Results. I like that title because it sounds almost unfair. If we all have access to the same AI models, the same tools, the same chat windows, why do the results differ so much? One person gets a useful plan. Another gets generic fluff. One team gets production-ready work. Another gets a pile of review comments. My argument today is simple: the difference is not mainly the model. The difference is the knowledge around the model. AI is becoming widely available. What is not evenly distributed is context, memory, judgement, standards, and the ability for agents to learn from what already happened.

The Familiar Moment

Let me start with a scene most of us recognize. Two people sit down with the same AI. They ask almost the same question. Maybe it is a developer asking for an implementation plan. Maybe it is a teacher asking for lesson material. Maybe it is a manager asking for a project brief. Both answers sound confident. Both answers are reasonable. But they are not the same. One fits the local reality. The other misses the hidden rules. It uses the wrong terminology, ignores a previous decision, suggests a tool the team already rejected, or solves a problem the organization does not actually have. The prompt was similar. The model was similar. The world behind the prompt was not.

Why Prompting Is Not Enough

When this happens, we often blame the prompt. We say: I should have written better instructions. Sometimes that is true. But after a while, better prompting becomes a strange way to run an organization. It means every person has to manually carry the company in their head and paste it into the chat again and again. Please remember our standards. Please remember how we talk to customers. Please remember the architecture. Please remember what the board decided last month. That is not a workflow. That is humans acting as memory sticks for machines. The more agents we use, the more expensive this becomes. We need the knowledge to live somewhere the agents can actually use.

The Technology Shock

Yesterday, at the AI strategy launch at Smæran, Hans Kári framed AI as a general-purpose technology, like electricity, steam, or the internet. That matters because general-purpose technologies do not land neatly. They do not simply remove a fixed percentage of jobs and call it a day. They change tasks, prices, expectations, and behavior. Some work is automated. Some work becomes more productive. Some work is affected indirectly. And there is uncertainty about timing. The important point for this audience is that we should not only ask, will AI change my profession? It will. The better question is: when the tools multiply, what keeps the work coherent?

Knowledge Work Is Judgement

Most knowledge work is not just producing words, code, slides, contracts, reports, or plans. Those are the visible outputs. Underneath them is judgement. Why did we choose this wording? Why did we avoid that vendor? Why do we onboard customers in this order? Why does this exception matter? Why is this number trusted and that one not trusted? An AI connected only to documents sees the output, but not the judgement that produced it. It sees the state of the work. It does not see the reasoning that made the state true. That is why generic AI can sound smart and still be locally wrong.

The Two Clocks

A useful way to think about this is two clocks. The first is the state clock: what is true now. Your CRM has a deal stage. Your codebase has a configuration. Your policy document has a sentence. Your project plan has a deadline. The second is the event clock: what happened, in what order, and why. Who changed the deal stage? What tradeoff led to that configuration? Why was that sentence written carefully? What happened the last time we missed a deadline? Organizations are very good at storing state. They are much worse at storing reasoning. But reasoning is exactly what agents need if we want them to work like colleagues instead of autocomplete.

Agents Raise The Stakes

This becomes more important when we move from chatbots to agents. A chatbot answers. An agent acts. It reads files, calls tools, opens pull requests, books meetings, updates tickets, sends summaries, and sometimes runs while we are not watching. When the system is only answering, missing context is annoying. When the system is acting, missing context becomes a governance problem. The agent needs to know what matters before it acts. It also needs to leave something behind after it acts. Otherwise every run is a disposable event. You pay for the work, but you do not accumulate learning.

What A Useful Knowledge Base Does

So when I say knowledge base, I do not mean a folder of PDFs. I do not mean a wiki nobody updates. I do not mean a vector database with mystery chunks. I mean a working memory that agents can interact with. Before the agent acts, it can search the relevant standards, decisions, examples, and prior solutions. While it acts, it can cite the fragments that shaped its plan. After it acts, it can write back what was learned: what changed, what worked, what failed, what should be reused next time. That loop is the difference between AI as a tool and AI as a compounding capability.

The Smallest Loop

The smallest version of the loop is almost boring, which is why it is powerful. Search before work. Act with context. Capture after work. Search before work. Act with context. Capture after work. If you do this for a day, it feels like documentation. If you do it for a month, it starts to feel like onboarding. If you do it for six months, it becomes an organizational memory that changes the quality of every future AI session. The same model now enters the room with a history. That is where different results begin.

Story: The Strategy Project

Today we heard a good example from Jaspur's work on a national AI strategy. A strategy process has many voices, many sources, and many months of context. Normally, a lot of that context disappears into meeting notes, folders, and the heads of the people who were there. In the Usable version, the project memory becomes part of the deliverable. The strategy is not only a final PDF. It is also a living memory layer that future work can ask questions of. That matters because Phase B does not have to restart from zero. The AI can work from the actual accumulated knowledge of the project, not from a generic idea of what a strategy should look like.

Story: The Game Project

Johann's game development story showed the same pattern in a very different domain. At first, AI can feel like a magic prompt box. Ask for a feature, get some code, push it around, ask again. But game development quickly exposes the limits of that. There is game feel, mechanics, visual language, constraints, experiments, things that were tried and rejected. The breakthrough was building a dedicated workspace for the project. The conversation stopped being a one-shot prompt and became a dialogue with memory. The agent could refer back to the world of the game, not just the latest message. Again: same AI, but now grounded in a richer local context.

Story: Two Operators

Brian's talk took it one level further: autonomous operators. One personal operator, one production operator. Same general engine, very different harness. The personal one can run with more freedom. The production one needs branch protection, observability, pull requests, and review. But the load-bearing part was the same: memory first. The operators read from Usable before acting and write results back afterwards. This is important because it moves us away from thinking about AI as one interface. Slack, IDE, browser, terminal, phone - those are surfaces. The durable thing is the memory layer underneath.

Decision Traces

Now for the cutting-edge part. The next step is not just storing more content. It is storing decision traces. A decision trace says: what did we observe, what constraints mattered, what tradeoff did we make, what action did we take, and what happened afterwards? This is different from a log. A log says what happened. A decision trace tries to preserve why it happened. That distinction matters because agents do not only need retrieval. They need precedent. They need to know how judgement has been applied in this organization before.

Accumulated Judgement

This is where the phrase knowledge management becomes too small, and organizational intelligence becomes more accurate. Your real asset is not only data. It is accumulated judgement. The way your team handles exceptions. The way your profession interprets constraints. The way your customers react when something changes. The way a senior person sees risk before a junior person can name it. If we can externalize even part of that judgement into a memory layer agents can use, we get a very different kind of AI deployment. Not a pilot that impresses people once, but a system that gets better because the organization keeps teaching it.

From Search To Simulation

A simple test is this: can your system answer what-if questions grounded in your own history? Not just, find me a document. Not just, summarize this meeting. But: what usually happens when we make this kind of change? Which stakeholders tend to care? What broke last time? Which decision pattern are we repeating? That is the shift from retrieval to simulation. We are still early, but this is the direction. The best knowledge bases for agents will not only help them find facts. They will help them reason over precedent.

Every Profession Has A Version

This is not only a software story. A teacher has examples that worked with a certain age group. A lawyer has clauses and negotiation rationale. A doctor has triage judgement and exception paths. A civil servant has policy intent and institutional constraints. A manager has promises, tradeoffs, and stakeholder history. A researcher has sources, failed hypotheses, and gaps. Every knowledge profession contains local judgement that generic AI cannot guess. The question is whether that judgement stays trapped in people's heads and scattered documents, or whether it becomes available to the agents that will increasingly participate in the work.

Personal Knowledge Bases

There is a personal version of this. Start by capturing the things you repeat. Your best explanations. Your preferred tone. The checklist you always rebuild. The decision you keep re-arguing with yourself. The examples that make your work recognizably yours. If your AI agent can read those before helping you, you will get less generic work. You will also become more consistent across tools. Today it might be ChatGPT. Tomorrow it might be Claude, Cursor, Hermes, or something else. The tool can change. Your memory should travel with you.

Team Knowledge Bases

There is also a team version. Capture the top five review comments you keep giving. Capture the standards that are obvious to insiders but invisible to newcomers. Capture the reason behind the architecture, not only the diagram. Capture the postmortem, but also the tradeoff that made the incident possible. Then connect the agent to that before it writes, plans, researches, or sends. This changes where quality control happens. Instead of correcting everything at review time, you move alignment earlier, into plan time. That is faster, but more importantly, it is calmer.

The Monday Playbook

So what should you do Monday morning? First, pick one repeated workflow. Not the whole organization. One workflow. Second, capture the standards and decisions that shape good work in that workflow. Third, make your AI agent search that memory before it acts. Fourth, ask it to cite the fragments that influenced important choices. Fifth, make the end of the task include a memory update: what changed, what was learned, what should the next person know? That is enough to start. You do not need a grand transformation program to stop throwing away learning.

Close

I want to leave you with the title again: Same AI, Different Results. In the age of AI, everyone can buy access to intelligence. That means advantage moves somewhere else. It moves to context. It moves to judgement. It moves to the memory layer that makes your agents act like they understand your world. If you are a knowledge professional, your work already contains the material. The question is whether your agents can interact with it. Build that memory layer now. Let it read before it acts. Let it write back after it acts. Same AI. Different knowledge base. Different results. Thank you.